

Here is a complete, beginner-friendly setup for a **C Calculator project** using gcc, make, and GitHub Actions.

In this setup, we separate our math logic from main.c so we can write unit tests that run automatically in CI.

1. Project Structure

Set up your repository files like this:

```
c-calculator/  
├── src/  
│   ├── calculator.h  
│   ├── calculator.c  
│   └── main.c  
├── tests/  
│   └── test_calculator.c  
├── .github/  
│   └── workflows/  
│       └── ci.yml  
└── Makefile
```

2. Code Files

A. Core Header & Logic (src/calculator.h & src/calculator.c)

```
// src/calculator.h  
#ifndef CALCULATOR_H  
#define CALCULATOR_H  
  
double add(double a, double b);  
double subtract(double a, double b);  
double multiply(double a, double b);  
double divide(double a, double b);  
  
#endif  
  
// src/calculator.c  
#include "calculator.h"  
#include <stdio.h>  
  
double add(double a, double b) { return a + b; }  
double subtract(double a, double b) { return a - b; }  
double multiply(double a, double b) { return a * b; }  
  
double divide(double a, double b) {  
    if (b == 0.0) {  
        printf("Error: Division by zero!\n");  
    }  
}
```

```

        return 0.0;
    }
    return a / b;
}

```

B. Interactive Main Program (src/main.c)

```

// src/main.c
#include <stdio.h>
#include "calculator.h"

int main() {
    printf("C Calculator Demo\n");
    printf("10 + 5 = %.2f\n", add(10, 5));
    printf("10 - 5 = %.2f\n", subtract(10, 5));
    printf("10 * 5 = %.2f\n", multiply(10, 5));
    printf("10 / 5 = %.2f\n", divide(10, 5));
    return 0;
}

```

3. Automated Unit Test File (tests/test_calculator.c)

We use standard C `assert()` statements. If any calculation gives an incorrect result, `assert()` returns a non-zero exit code (exit code 1), causing GitHub Actions to fail.

```

// tests/test_calculator.c
#include <assert.h>
#include <stdio.h>
#include "../src/calculator.h"

int main() {
    // Test Addition
    assert(add(2.0, 3.0) == 5.0);

    // Test Subtraction
    assert(subtract(5.0, 2.0) == 3.0);

    // Test Multiplication
    assert(multiply(4.0, 2.5) == 10.0);

    // Test Division
    assert(divide(10.0, 2.0) == 5.0);
    assert(divide(5.0, 0.0) == 0.0); // Edge case: division by zero

    printf("✅ All calculator test cases passed successfully!\n");
    return 0;
}

```

```
}
```

4. The Makefile (Makefile)

A Makefile standardizes compilation commands so students (and CI runners) can compile and test using clean commands.

```
CC = gcc
```

```
CFLAGS = -Wall -Isrc
```

```
# Default target: build main program
```

```
all:
```

```
    $(CC) $(CFLAGS) src/calculator.c src/main.c -o calculator
```

```
# Target to compile and run tests
```

```
test:
```

```
    $(CC) $(CFLAGS) src/calculator.c tests/test_calculator.c -o test_runner
```

```
    ./test_runner
```

```
clean:
```

```
    rm -f calculator test_runner
```

5. GitHub Actions CI Pipeline (.github/workflows/ci.yml)

This YAML workflow spins up an Ubuntu Linux runner, installs gcc and make, compiles the binary, and executes your test runner.

```
name: C Calculator CI Pipeline
```

```
on:
```

```
  push:
```

```
    branches: [ "main" ]
```

```
  pull_request:
```

```
    branches: [ "main" ]
```

```
jobs:
```

```
  build-and-test:
```

```
    runs-on: ubuntu-latest
```

```
    steps:
```

```
      # Step 1: Download repo code onto runner
```

```
      - name: Checkout Code
```

```
        uses: actions/checkout@v4
```

```
      # Step 2: Install compiler and build tools
```

- name: Install Build Tools
run: |
 sudo apt-get update
 sudo apt-get install -y gcc make

Step 3: Compile main application

- name: Compile Application
run: make all

Step 4: Run test suite (CI Check)

- name: Run Test Cases
run: make test

6. Testing It Out (Teaching Demonstration)

1. **Verify locally first**

Local terminal

Run your Makefile commands locally to make sure everything compiles: You should see:

✅ All calculator test cases passed successfully!

2. **Push code to GitHub**

Git commit & push

Commit all files and push them to main:

3. **Observe the green build**

GitHub UI

Go to the Actions tab on your GitHub repository. Click on C Calculator CI Pipeline to see gcc compile your code and execute make test live.

4. **Demonstrate a failing CI build**

Student Challenge

Change add() in src/calculator.c to return a - b instead of a + b, then push to GitHub. The workflow will fail on assert(add(2.0, 3.0) == 5.0) and turn red ❌.